STMemory: Fixed-Query Spatiotemporal Memory for World Models

WCM Project

October 2026

Abstract

A world model must often judge the present from evidence that is no longer in view. In robotic manipulation, two moments that look identical can lie at very different stages of a task, and a model that sees only the latest frames cannot tell them apart. Giving the model access to a longer history usually means feeding it more tokens, so memory and computational cost grow together. We present SpaceTime Memory (STMemory), a visual memory module that breaks this coupling. STMemory samples observations sparsely across an adjustable time span, relates them through factorized spatiotemporal attention, and distils them into a fixed set of learned queries, so the representation passed to the predictor has the same size however far back the memory reaches. Built into a JEPA-style world critic for real-robot manipulation, STMemory reduces out-of-distribution value error by about 80% relative to the original model, outperforms VIP and IQL value learners given the same history, and raises the correlation between predicted and true returns from 0.21 to 0.86. Doubling the memory span lowers error by a further 20% at identical cost, and the resulting value estimate separates successful from failed episodes with perfect accuracy. These results show that how far a world model remembers matters as much as how much it encodes, and that fixed-query memory delivers long temporal reach at constant cost.

1 Introduction

Imagine a robot arm hovering above a button. From a single frame, it is impossible to say whether the arm is about to press the button or has just finished pressing it, yet the two situations call for opposite judgments about how the task is going. World models face this ambiguity constantly. They learn to predict how an environment evolves so that an agent can plan and evaluate its behaviour [1–3], but each camera frame reveals only part of the underlying state. Reliable prediction therefore depends on memory, the ability to bring earlier evidence to bear on the current decision.

Existing world models provide memory in three main ways (Fig. 1). Many recent models condition on a short window of recent frames, which keeps computation small but discards anything that happened earlier [4]. Video transformers instead encode every frame as a dense set of spatiotemporal tokens [5–7]. This preserves direct access to the past, but the number of tokens, and with it the cost of every downstream computation, grows with the length of the history. Recurrent world models compress the past into a running latent state [1–3], which is compact but forces information through a sequential bottleneck in which earlier observations can only be recovered indirectly.

We argue that temporal reach and representation size should be controlled separately. A model should be able to look further into the past without paying for it in tokens, and it should read earlier frames directly rather than through a chain of state updates. This is the principle behind SpaceTime Memory (STMemory). STMemory samples a small number of observations spread across a chosen time span, encodes them with a shared vision transformer [8], relates them through attention along the time axis, and compresses the result into a fixed set of learned queries in the spirit of the Perceiver [9]. Widening the span changes only which frames are sampled. The number of tokens, the size of the memory and the cost of the predictor remain unchanged.

The importance of this decoupling is practical as well as conceptual. Robots operate at high frame rates, and the events that determine the outcome of a task can lie seconds in the past. A memory whose cost is fixed by design can reach these events without slowing down prediction, and its compact output can be attached to an existing world model without changing the predictor.

We build STMemory into the World Critic Model (WCM) [4], a JEPA-style world model that jointly predicts future latent states and task values, and evaluate it on real-robot manipulation. Our contributions are as follows.

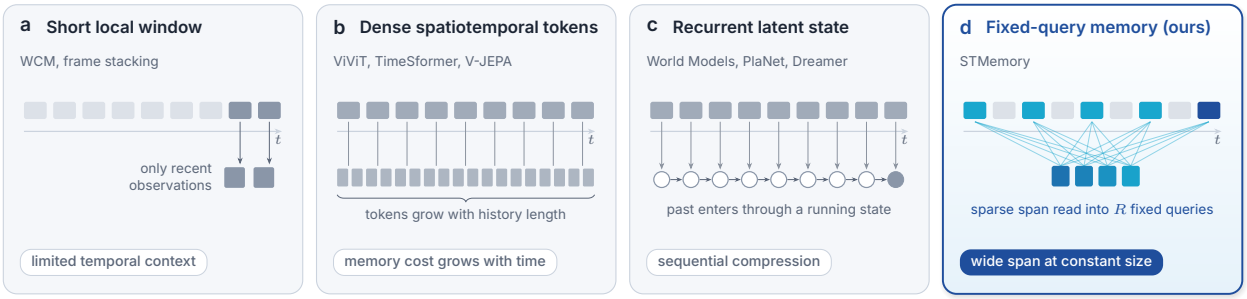


Figure 1. Four ways to give a world model memory. **a**, A short window of recent frames. **b**, Dense tokens for every frame. **c**, A recurrent latent state. **d**, STMemory reads a sparse sample of the past into a fixed number of learned queries.

- We introduce STMemory, a spatiotemporal memory module that decouples the temporal reach of a world model from the size of its representation.
- We show that STMemory reduces out-of-distribution value error by about 80% relative to the original WCM and outperforms VIP [10] and IQL [11] value learners that receive the same visual history, as well as a four times larger single-frame $\pi^*_{0.6}$ value model [12].
- Through a controlled ablation, we show that doubling the memory span at fixed frame and query counts lowers error by a further 16 to 24% across seeds.
- We show that the memory-augmented value estimate separates successful from failed episodes without a dedicated classification head.

2 Related Work

World models and predictive representations. World models learn latent dynamics for simulation, planning and policy learning [1–3], and recent work extends them to interactive generation [13] and to coupled video and action prediction for robot control [14, 15]. Joint-Embedding Predictive Architectures learn by predicting target representations rather than pixels [7, 16], which focuses the model on predictable structure. WCM applies this principle to robotic value estimation by predicting future latents together with task values [4]. STMemory is complementary to these predictive objectives and changes only the visual evidence available to the predictor.

Temporal modelling and token compression. Video transformers factorize attention over space and time to make long clips tractable [5, 6]. Learned query bottlenecks compress large token sets into a small latent array, as in the Perceiver [9], TokenLearner [17], the Perceiver Resampler of Flamingo [18] and the Q-Former of BLIP-2 [19]. STMemory combines factorized temporal attention with a query bottleneck and applies them to a sparse sample of the past, so that the span of memory can be enlarged while the bottleneck stays fixed.

Value learning for robot manipulation. Visual value functions provide progress estimates and rewards for robot learning. VIP learns a goal-conditioned embedding whose distances act as values [10], and IQL learns value functions offline through expectile regression [11]. $\pi^*_{0.6}$ trains a distributional value function on a vision-language backbone and uses its advantage estimates to improve a robot policy [12]. These methods treat temporal context as part of the input design. STMemory instead provides a dedicated memory representation, and we compare against VIP and IQL under the same visual history and against the single-frame $\pi^*_{0.6}$ value function.

3 Method

3.1 Predictive world model

At step t, the agent receives camera observations $x_{\leq t}$ from C synchronized views and a task instruction ℓ. Following JEPA [7, 16], a context encoder maps the available evidence to a representation c_t, and a predictor

forecasts the latent of the next observation rather than its pixels. WCM [4] attaches a state-value head $\hat{V}_t = v_\omega(c_t)$, an action-value head and an action-conditioned dynamics head,

$$\hat{z}_{t+1} = z_t + \Delta_\psi\big(c_t, a_t\big), \tag{1}$$

where z_t is the current visual latent and a_t the action. Predicting an increment anchors the forecast to the present state. Training minimizes

$$\mathcal{L} = \big(\hat{V}_t - G_t\big)^2 + \lambda_{\text{dyn}} \left\| \hat{z}_{t+1} - \text{sg}(z_{t+1}) \right\|_2^2 + \lambda_{\text{reg}}\, \mathcal{L}_{\text{reg}}, \tag{2}$$

where G_t is the normalized return, sg stops gradients through the target latent and $\mathcal{L}_{\text{reg}}$ is a Gaussian embedding regularizer that prevents representational collapse. Future observations serve only as targets. STMemory replaces the visual evidence entering c_t and leaves this objective unchanged.

3.2 Sparse memory sampling

STMemory represents the past by T observations drawn from a window of k frames. Given offsets $k = \delta_1 > \delta_2 > \cdots > \delta_T = 0$, the observation memory is

$$\mathcal{X}_t^{(k)} = \big(x_{u_1}, \ldots, x_{u_T}\big), \qquad u_i = \max\big(0,\, t - \delta_i\big), \tag{3}$$

ordered from oldest to current, with indices before the episode start clamped to its first frame. The span k sets how far the memory reaches, while T sets how many observations are processed. Because the two are independent, the span can grow without increasing the input size. The memory is rebuilt from within-episode observations at every prediction.

3.3 Spatiotemporal encoding

A shared vision transformer ϕ encodes every view of every sampled observation into one class token and a grid of patch tokens [8]. For time slot i, view v and token j, the input representation is

$$E_{i,v,j} = \phi\big(x_{u_i}^{(v)}\big)_j + e_i^{\text{time}} + e_v^{\text{cam}}, \tag{4}$$

where e_i^{time} and e_v^{cam} are learned slot and camera embeddings. Indexing time embeddings by slot rather than by elapsed frames allows the same parameters to serve any span.

Temporal relations are modelled by attention along the time axis at each spatial position, in the factorized form used by video transformers [5, 6]. For the sequence $Y_{v,j} = (E_{1,v,j}, \ldots, E_{T,v,j})$, each layer applies

$$Y_{v,j} \leftarrow Y_{v,j} + \text{MHA}\big(\text{LN}(Y_{v,j})\big), \qquad Y_{v,j} \leftarrow Y_{v,j} + \text{FFN}\big(\text{LN}(Y_{v,j})\big), \tag{5}$$

where MHA is multi-head self-attention, LN layer normalization and FFN a feed-forward block. Each patch thereby gathers information from the same location at every sampled time, which captures motion and state changes without an attention matrix over all tokens. The outputs are flattened into a sequence $S_t \in \mathbb{R}^{N \times d}$ with $N = TCP$ tokens of width d, where P is the number of tokens per view.

3.4 Fixed-query compression

A set of R learned queries $\mathbf{Q} \in \mathbb{R}^{R \times d}$ reads the spatiotemporal sequence through cross-attention [9],

$$\mathbf{Q} \leftarrow \mathbf{Q} + \text{MHA}\big(\text{LN}(\mathbf{Q}), \text{LN}(S_t)\big), \qquad \mathbf{Q} \leftarrow \mathbf{Q} + \text{FFN}\big(\text{LN}(\mathbf{Q})\big), \tag{6}$$

where queries provide the attention queries and S_t the keys and values. After the final layer, a normalization and linear projection Π produce the memory

$$\mathcal{M}_t^{(k)} = \Pi\Big(\Psi_\theta\big(\phi(\mathcal{X}_t^{(k)})\big)\Big) \in \mathbb{R}^{R \times d}, \tag{7}$$

where Ψ_θ collects the embedding, temporal and query layers. The size of $\mathcal{M}_t^{(k)}$ depends only on R and d. All R memory tokens enter the context encoder together with the instruction embedding, $c_t = C_\theta(\mathcal{M}_t^{(k)}, e_\ell)$, and their mean serves as the visual latent z_t in Eq. (1).

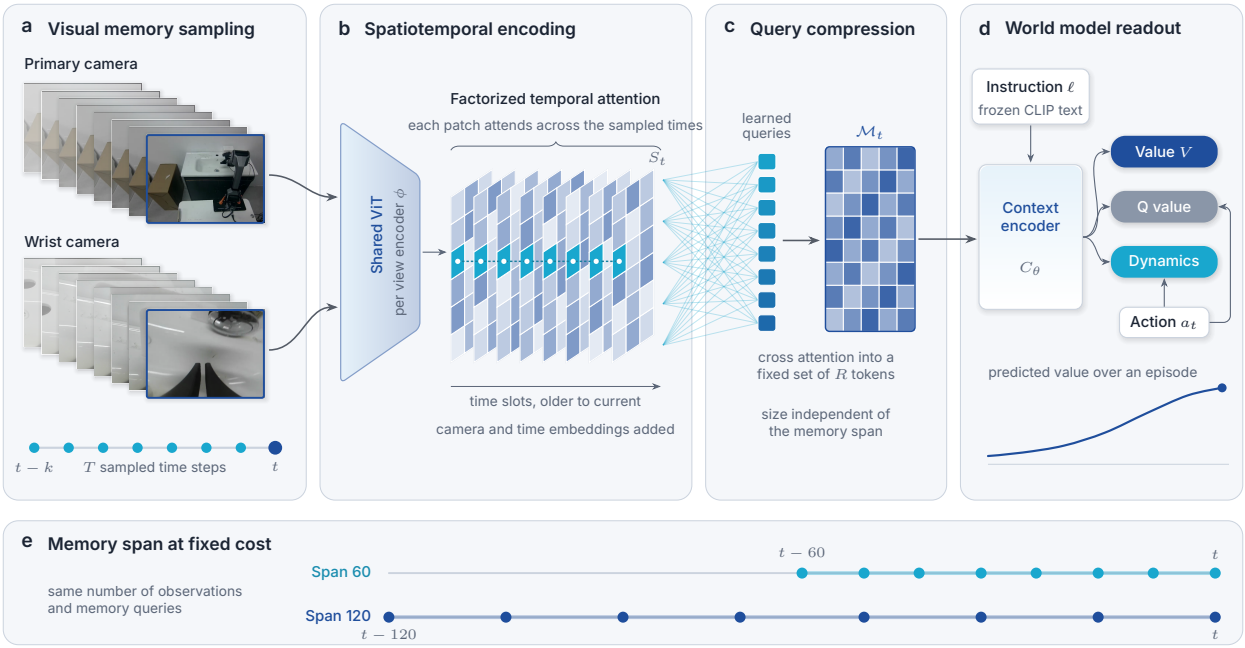


Figure 2. STMemory architecture. **a**, Observations from two cameras are sampled across the memory span. **b**, A shared ViT encodes each view, and factorized temporal attention links each patch across time. **c**, Learned queries compress the token sequence into a fixed memory $\mathcal{M}_t$. **d**, The memory conditions the WCM value, action-value and dynamics heads. **e**, Two memory spans with identical cost.

3.5 Outcome classification from value

A value function that tracks task progress should also indicate whether an episode will succeed. We classify episode i as successful when its predicted value exceeds a threshold,

$$\hat{y}_i = \mathbf{1}\big[\hat{V}_i > \tau_{-i}\big], \tag{8}$$

where τ_{-i} is chosen on all other episodes to maximize balanced accuracy and is then applied to episode i. No classification head or additional training is involved.

3.6 Computational cost

Temporal attention scales as $O(CPT^2)$ and query cross-attention as $O(RTCP)$. Neither term depends on the span k, and the predictor always receives R memory tokens. Extending the reach of memory is therefore free in both computation and representation size, and the only cost of memory is set by the choice of T and R.

4 Experiments

4.1 Experimental setup

Task and datasets. We evaluate on a real-robot manipulation task in which a six-axis arm with a gripper presses a button following the instruction *press the toilet button*. Each episode provides synchronized primary and wrist camera streams at 30 Hz. The training set contains 177 episodes collected in three sessions in the same scene and includes both successful and failed attempts, split into 159 training and 18 validation episodes. We test on two held-out sets. The in-distribution test set contains 145 new episodes from the training scene. The out-of-distribution (OOD) test set contains 125 episodes recorded after the scene layout was changed by placing a board over the work surface, which alters the appearance and geometry of the environment. The OOD set is our primary benchmark.

Baselines. We compare against the original WCM [4] trained with its published recipe, and against two established value-learning methods. VIP [10] learns a value-implicit embedding from successful trajectories, and IQL [11] learns a critic through temporal-difference learning with expectile regression. Both receive exactly the same eight-frame history with a span of 120 frames, the same two cameras, the same instruction and the current robot state as STMemory-120, using frozen ResNet-18 [20] image features and CLIP text

Table 1. Value prediction on the out-of-distribution and in-distribution test sets. Mean ± standard deviation over three seeds. †Evaluated on the 21,832 OOD frames available to the WCM observation window. *Single seed.

		Out-of-distribution			In-distribution	
Method	Visual history	MSE ↓	Pearson ↑	Centered MSE ↓	MSE ↓	Pearson ↑
WCM [4]†	single frame	0.124 ± 0.024	0.21 ± 0.06	0.066 ± 0.007	0.0384 ± 0.0020	0.38 ± 0.04
$\pi^*_{0.6}$ value [12]†,*	single frame	0.075	0.51	0.026	–	–
VIP [10]*	8 frames, span 120	0.071	0.47	0.055	–	–
IQL critic [11]*	8 frames, span 120	0.044	0.43	0.034	–	–
STMemory-60	8 frames, span 60	0.025 ± 0.008	0.84 ± 0.01	0.019 ± 0.004	0.0099 ± 0.0004	0.87 ± 0.00
STMemory-120	8 frames, span 120	**0.020 ± 0.007**	**0.86 ± 0.01**	**0.015 ± 0.004**	0.0093*	0.87

features [21]. We further compare against the distributional value function of $\pi^*_{0.6}$ [12], which we re-implement and train on an earlier collection from the training scene. It reads the current frame of both cameras at 384×384 resolution together with the robot state written as text, combines a SigLIP image encoder [22] with the 270M-parameter Gemma 3 language model [23], and predicts a distribution over 201 value bins. With about 670M parameters it is four times the size of WCM, and its training target adds a bonus to the return in the middle of each episode. We denote our models by their memory span as STMemory-60 and STMemory-120.

Metrics. Value prediction is measured by mean squared error (MSE) against the normalized return, Pearson correlation, which reflects whether states are ranked correctly by expected return, and centered MSE, which removes a constant offset. All comparisons use identical episode and frame endpoints. Confidence intervals come from 20,000 paired bootstrap resamples over episodes. Outcome classification is evaluated on the 18 validation episodes, 13 successful and five failed, using the earliest frame available to all models.

Implementation details. The vision encoder is ViT-B/16 at 224×224 resolution, and STMemory uses eight observations, two temporal layers, two query layers and 64 queries of width 384. Training proceeds in two stages. In the joint stage, the newly added memory layers are trained with a tenfold larger learning rate while only the top four ViT blocks are unfrozen. This lets the randomly initialized memory adapt quickly without disturbing the pretrained visual features. A short full fine-tuning stage at a lower learning rate then aligns the entire backbone with the memory. All results are averaged over three random seeds unless stated otherwise. Full hyperparameters and the compute environment are listed in Appendix A.

4.2 Main results

Table 1 and Fig. 3 summarize value prediction on both test sets. On the OOD benchmark, STMemory-60 reduces value MSE by 79.8% relative to the original WCM and raises Pearson correlation from 0.21 to 0.84, with every per-seed improvement significant. STMemory-120 performs best on every metric. Compared with VIP and IQL, which receive the same history, it lowers MSE by 70.3% and 52.0% respectively, and both paired confidence intervals lie entirely below zero.

The $\pi^*_{0.6}$ value model separates the effect of model size from that of memory. Its larger backbone lowers the error of WCM, yet with only the current frame it reaches a correlation of 0.51. On the same frames, STMemory-60 reduces its MSE by 61.5% on average over two seeds and raises correlation to 0.84, and every paired confidence interval excludes zero. Adding memory at the size of WCM is worth more than a fourfold larger single-frame model.

The gap in correlation is especially informative. VIP, IQL and the $\pi^*_{0.6}$ value model reach correlations of at most 0.51, so their estimates capture only part of the variation in return across states. STMemory exceeds 0.84 on both test sets, so its estimates order states closely by their true returns. The centered MSE confirms that the gain does not come from a constant calibration offset. On the in-distribution test set, STMemory lowers MSE by roughly a factor of four relative to WCM, showing that the benefit of memory holds in the training scene as well as under distribution shift.

4.3 Ablation on memory span

To isolate the role of temporal reach, we compare STMemory-60 and STMemory-120, which share the architecture, the number of observations, the 64-query bottleneck, the initialization and the training schedule.

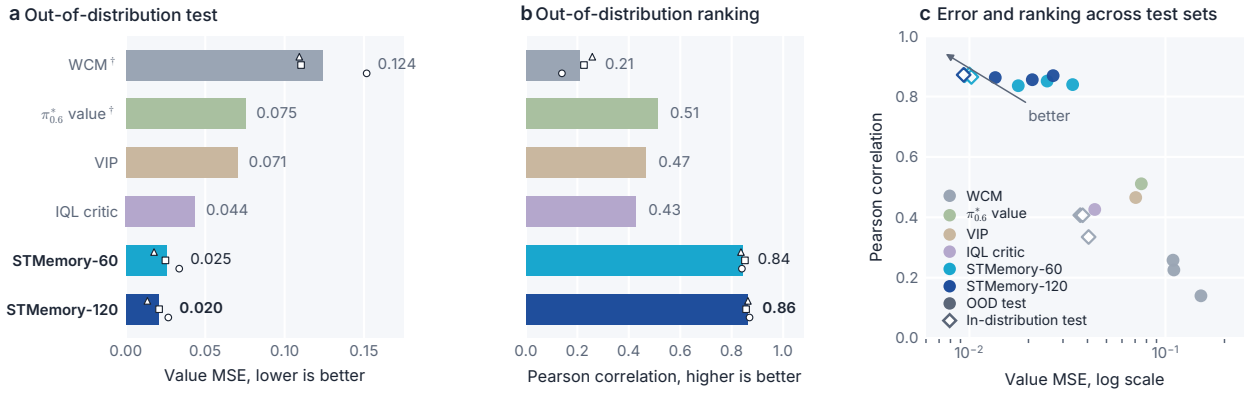


Figure 3. Value prediction performance. **a**,**b**, OOD value MSE and Pearson correlation; markers show individual seeds. †Evaluated on the frames available to the WCM observation window. **c**, Each run placed by its error and correlation on both test sets.

Table 2. Effect of memory span on OOD value prediction. Both variants use eight observations and 64 queries. The last column gives the 95% confidence interval of the Pearson gain.

	MSE ↓			Pearson ↑		
	Span 60	Span 120	Reduction (%)	Span 60	Span 120	Gain CI
Seed 1	0.0336	**0.0267**	20.5	0.840	**0.869**	[0.018, 0.043]
Seed 2	0.0249	**0.0209**	16.0	0.851	**0.856**	[-0.006, 0.016]
Seed 3	0.0177	**0.0135**	23.7	0.836	**0.863**	[0.012, 0.044]
Mean	0.0254	**0.0204**	20.1	0.842	**0.863**	–

They differ only in the offsets used to sample the past (Appendix B). Table 2 and Fig. 4 report the paired comparison on the OOD set.

Doubling the span reduces MSE for every seed, by 20.1% on average, and all three confidence intervals lie below zero. Correlation increases for every seed, with significant gains for two of them. Because the wider span covers twice the time with the same eight observations, these results show that distant observations carry information that remains useful after compression into the fixed memory. Reach, rather than sampling density, is the more valuable resource in this setting.

Efficiency. Table 3 lists the cost of adding memory. STMemory increases the parameter count by 3% and processes eight observations in 12.9 ms per sample, compared with 7.0 ms for the single-frame WCM. This corresponds to more than 75 predictions per second, well above the 30 Hz camera rate. Since the predictor always receives 64 memory tokens, the span ablation above comes at no additional cost.

4.4 Outcome classification

We next ask whether the value estimate encodes task outcome. Using the rule in Eq. (8) at the earliest common frame, STMemory-120 classifies every validation episode correctly for all three seeds, and STMemory-60 misses a single failure in one seed (Table 4). The original WCM reaches 66.7% accuracy, below the 72.2% obtained by always predicting success. The AUROC of the raw value is 1.0 for both STMemory variants, which means that every successful episode receives a higher value than every failed one.

Fig. 5b shows why. For WCM, the held-out margins of successful and failed episodes overlap, and many successful episodes fall below the threshold. With STMemory, successes and failures form two clearly separated groups on either side of the decision boundary. The value learned with memory therefore separates outcomes directly, and a single threshold suffices to turn it into a reliable outcome classifier.

4.5 Qualitative results

Fig. 6 shows predicted values over two complete episodes. In the successful OOD episode, the value of STMemory-120 rises steadily from -0.78 to nearly zero as the arm approaches and presses the button. In the failed episode, the value of STMemory-60 stays low through the first half while the arm fails to make progress, and it remains well below the level of the successful episode at the end. The estimates thus follow

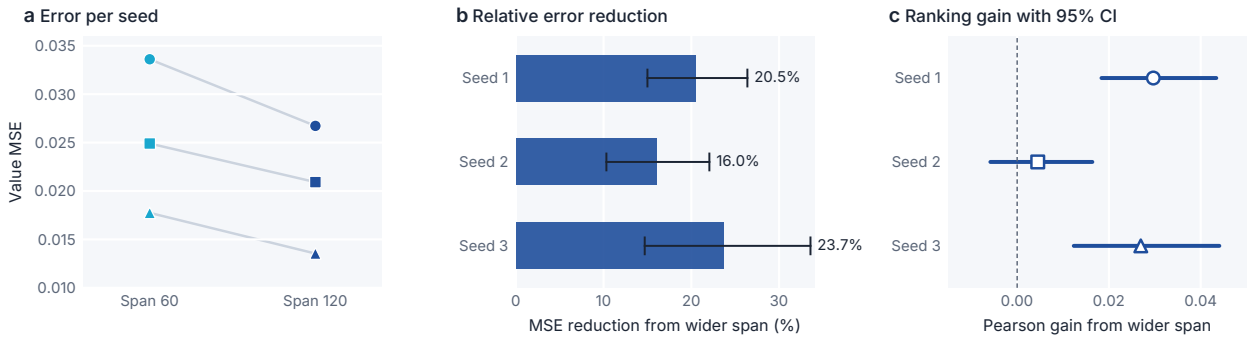


Figure 4. A wider memory span improves value prediction at fixed cost. **a**, OOD MSE of each seed for the two spans. **b**, Relative MSE reduction with 95% confidence intervals. **c**, Pearson gain with 95% confidence intervals.

Table 3. Model size and inference cost, measured at batch size four on one GPU.

Model	Parameters	Observations	Memory tokens	Latency per sample
WCM	170.6 M	1	1	7.0 ms
STMemory-60 and STMemory-120	176.1 M	8	64	12.9 ms

task progress in a smooth and interpretable way. Full videos are provided as Supplementary Videos S1 and S2.

Table 4. Success and failure classification from the value estimate. 18 validation episodes, mean ± standard deviation over three seeds.

Model	Accuracy (%)	Balanced accuracy (%)	Failure recall (%)	AUROC
WCM	66.7 ± 5.6	76.9 ± 3.8	100.0 ± 0.0	0.677 ± 0.096
STMemory-60	98.1 ± 3.2	96.7 ± 5.8	93.3 ± 11.5	1.000 ± 0.000
STMemory-120	**100.0 ± 0.0**	**100.0 ± 0.0**	**100.0 ± 0.0**	**1.000 ± 0.000**

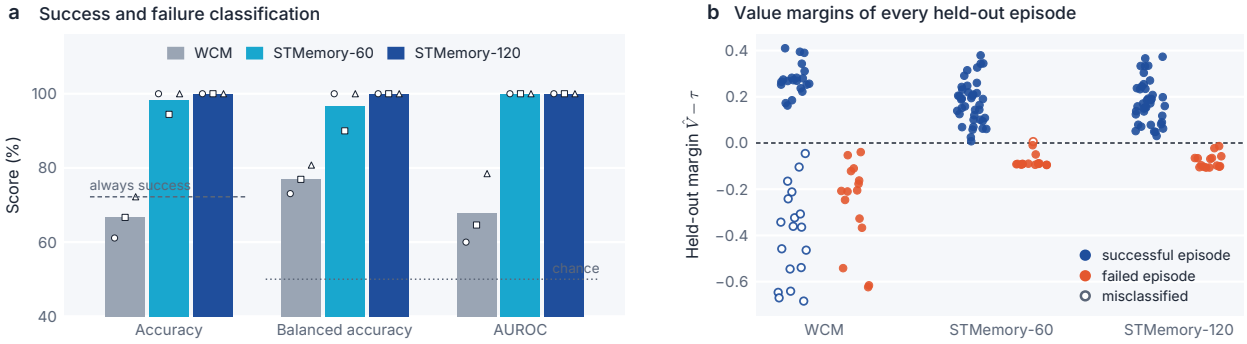


Figure 5. Outcome classification from the value estimate. **a**, Classification scores; markers show seeds. **b**, Held-out value margins of all episodes and seeds; positive margins predict success.

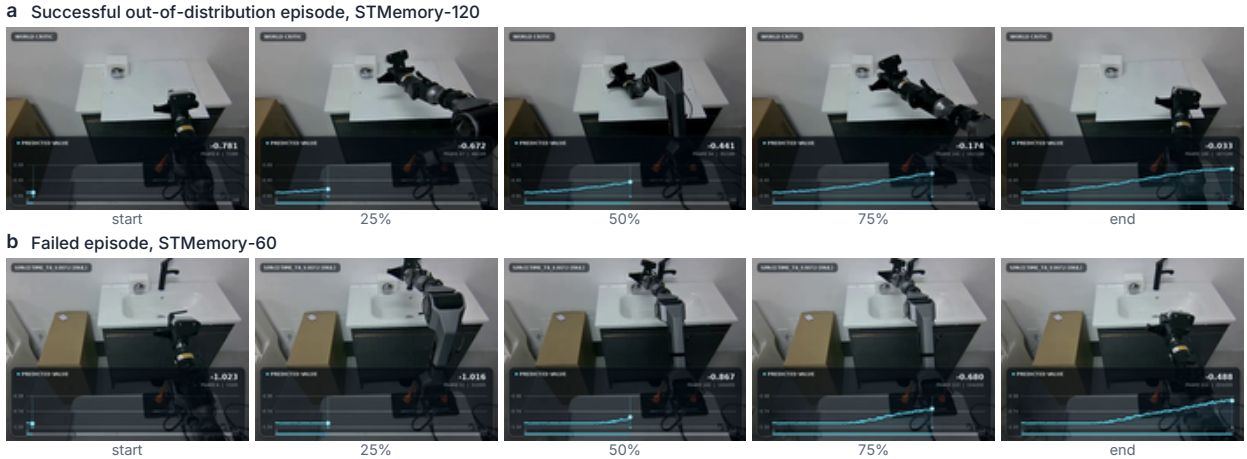


Figure 6. Predicted value over real robot episodes. Frames at five stages of **a**, a successful OOD episode and **b**, a failed episode. The overlay shows the predicted value up to the current frame.

5 Conclusion

We presented STMemory, a spatiotemporal memory module that lets a world model reach further into the past without enlarging its representation. By sampling observations sparsely across a time span, relating them through factorized temporal attention and compressing them into a fixed set of learned queries, STMemory decouples temporal reach from cost. In real-robot manipulation, it reduced out-of-distribution value error by about 80% relative to the original WCM, outperformed VIP and IQL under the same visual history as well as a four times larger single-frame value model, and produced value estimates that separate successful from failed episodes. A wider span improved accuracy further at identical cost. These findings identify temporal reach as a central design variable for memory in world models. Promising directions include learning where in the past to sample, extending the memory with a persistent state across episodes, and applying STMemory to planning and control in other world-model architectures, where its fixed-size interface allows integration without changes to the predictor.

A Implementation details

Table 5. Hyperparameters.

Architecture	
Vision encoder	ViT-B/16, 224×224 input, projection to width 384
Text encoder	frozen CLIP ViT-B/32 text tower, two fusion layers
Context trunk	six blocks, eight heads, feed-forward ratio four, dropout 0.1
Dynamics head	three blocks
STMemory	eight observations, two cameras, two temporal layers, two query layers
Memory	64 queries of width 384, eight heads, feed-forward ratio two
Residual scaling	learned LayerScale initialized to 10^{-3}
Optimization	
Optimizer	AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 10^{-3}
Precision and clipping	bf16, gradient norm 1.0
Initialization	WCM checkpoint trained on the same training split
Joint stage	ten epochs, learning rate 10^{-5}, memory layers $10\times$, 150 warm-up steps, top four ViT blocks trainable
Fine-tuning stage	three epochs, learning rate 2×10^{-6}, 50 warm-up steps, full backbone trainable
Schedule	cosine decay, batch size four

Table 6. Compute environment.

Operating system	Ubuntu 22.04.5 LTS
Processor	AMD EPYC 7K62, 48 cores
Memory	252 GiB
GPUs	NVIDIA RTX PRO 5000 Blackwell and NVIDIA RTX 5880 Ada, 48 GiB each
Software	Python 3.12, PyTorch 2.7.1, CUDA 12.6, Transformers 5.15

Baseline adaptation. VIP and IQL use frozen ResNet-18 features for both cameras at all eight sampled time steps, CLIP text features and a trainable projection and value network. Both are trained for 1,000 updates with batch size 128, AdamW and learning rate 3×10^{-4}, and the checkpoint with the lowest validation MSE is selected. The VIP value head reads the value-implicit embedding, and the IQL critic uses twin Q networks with expectile 0.7 and no discounting, matching the return definition.

The $\pi^*_{0.6}$ value model uses a SigLIP So400m/14 image encoder at 384×384 resolution for each camera and a Gemma 3 270M language model, which reads the instruction and the discretized robot state as text, and a value head over 201 bins. Its training target is the normalized return with a failure penalty and a mid-episode bonus of scale 0.3. A single model is available. Its predictions on all OOD frames are produced with its own inference code and scored against the same normalized returns as every other model.

B Sampling offsets

The offsets δ_i in Eq. (3), in frames before the current step, are

$$\begin{aligned} \text{STMemory-60} \quad & \{60, 51, 43, 34, 26, 17, 9, 0\}, \\ \text{STMemory-120} \quad & \{120, 103, 86, 69, 51, 34, 17, 0\}. \end{aligned}$$

C Per-seed results and training dynamics

Table 7. OOD value prediction for each seed.

	Seed 1		Seed 2		Seed 3	
Model	MSE	Pearson	MSE	Pearson	MSE	Pearson
WCM	0.1517	0.140	0.1104	0.226	0.1093	0.258
STMemory-60	0.0336	0.840	0.0249	0.851	0.0177	0.836
STMemory-120	0.0267	0.869	0.0209	0.856	0.0135	0.863

Fig. 7 shows training and validation curves. STMemory starts from a lower training value loss than WCM and reaches a several times lower validation value MSE within the first epochs. Latent losses are computed against each model's own encoder and are therefore compared within, not across, models.

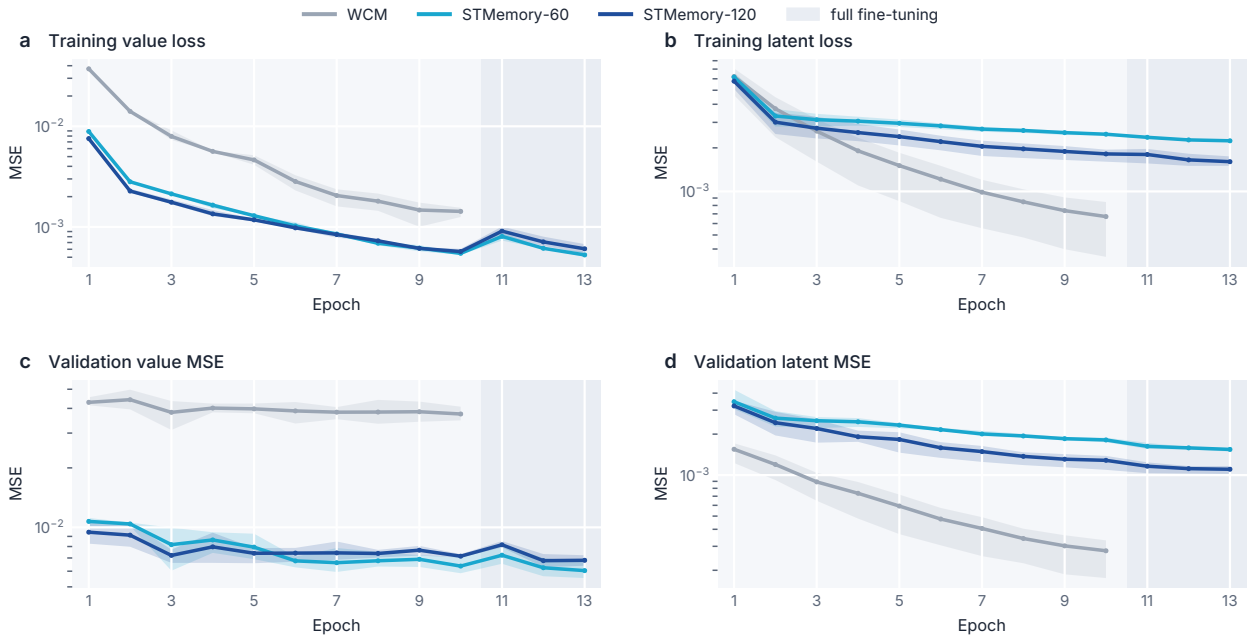


Figure 7. Training dynamics. Seed means with seed ranges for training losses (**a**,**b**) and validation errors (**c**,**d**). The shaded region marks full fine-tuning.

References

[1] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. doi: 10.48550/arXiv.1803.10122. URL https://arxiv.org/abs/1803.10122.

[2] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2555–2565. PMLR, 2019. URL https://proceedings.mlr.press/v97/hafner19a.html.

[3] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025. doi: 10.1038/s41586-025-08744-2.

[4] Senyu Fei, Xiaopeng Yu, Siyin Wang, Xianzhong Zhao, Jingjing Gong, and Xipeng Qiu. Wcm: A world critic model for vision-language-action reinforcement learning. *arXiv preprint arXiv:2607.29613*, 2026.

[5] Anurag Arnab, Mostafa Dehghani, Georg Heigold, Chen Sun, Mario Lučić, and Cordelia Schmid. ViViT: A video vision transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.

[6] Gedas Bertasius, Heng Wang, and Lorenzo Torresani. Is space-time attention all you need for video understanding? In *International Conference on Machine Learning*, 2021.
[7] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024. doi: 10.48550/arXiv.2404.08471. URL https://arxiv.org/abs/2404.08471.
[8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
[9] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, et al. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning (ICML)*, 2021.
[10] Yecheng Jason Ma, Shagun Sodhani, Dinesh Jayaraman, Osbert Bastani, Vikash Kumar, and Amy Zhang. VIP: Towards universal visual reward and representation via value-implicit pre-training. In *International Conference on Learning Representations*, 2023.
[11] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations*, 2022.
[12] Physical Intelligence. $\pi^*_{0.6}$: a VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
[13] Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, et al. Genie: Generative interactive environments. *arXiv preprint arXiv:2402.15391*, 2024. doi: 10.48550/arXiv.2402.15391. URL https://arxiv.org/abs/2402.15391.
[14] Kai Wang, Zhaopeng Gu, Yixiang Chen, Yuan Xu, Qisen Ma, Jiabing Yang, Zhaowen Li, Yan Huang, Liang Wang, and Peng Su. DIM-WAM: World-action modeling with diverse historical event memory. *arXiv preprint arXiv:2606.27677*, 2026. doi: 10.48550/arXiv.2606.27677. URL https://arxiv.org/abs/2606.27677.
[15] Yinghua Zhou, Junjie Ye, Yiqi Zhao, Hao Dong, Celina Shiyu Wang, Ruohai Ge, Tingyi Yang, Basile Van Hoorick, Gaurav Sukhatme, Vitor Guizilini, and Yue Wang. Rolling-WAM: World action models with rolling imagination. *arXiv preprint arXiv:2609.30247*, 2026. doi: 10.48550/arXiv.2609.30247. URL https://arxiv.org/abs/2609.30247.
[16] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023. URL https://arxiv.org/abs/2301.08243.
[17] Michael S. Ryoo, AJ Piergiovanni, Anurag Arnab, Mostafa Dehghani, and Anelia Angelova. TokenLearner: Adaptive space-time tokenization for videos. In *Advances in Neural Information Processing Systems*, 2021.
[18] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, 2022.
[19] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning*, 2023.
[20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
[21] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.
[22] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
[23] Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.